

ATIVIDADE AVALIATIVA – ESTRUTURAS DE DADOS

Arrays, Matrizes, Algoritmos de Ordenação e Busca

Alunos: Jorge Luis Soares Dos Santos rgm:45987017

Felipe Falcão Campelo rgm:43940633

Parte 1 – Pesquisa: Bubble Sort e Quick Sort

Bubble Sort

Como funciona / lógica de ordenação:

- Percorre o array repetidamente comparando pares de elementos adjacentes.
- Sempre que um par está fora de ordem, os dois elementos são trocados de posição.
- A cada passagem completa, o maior valor ainda não ordenado "flutua" até sua posição final, no fim do trecho não ordenado.
- O processo se repete até que uma passagem inteira não realize nenhuma troca, o que indica que o array está ordenado.
- Estrutura: dois laços aninhados — o externo controla o número de passagens, o interno compara e troca elementos adjacentes.

Complexidade — Melhor caso: $O(n)$, quando o array já está ordenado e a implementação usa uma flag de parada antecipada ("swapped").

Complexidade — Caso médio: $O(n^2)$, pois em média metade das comparações resulta em troca.

Complexidade — Pior caso: $O(n^2)$, quando o array está em ordem totalmente inversa — todas as comparações resultam em troca.

Vantagens:

- Implementação extremamente simples e intuitiva.
- Algoritmo estável (mantém a ordem relativa de elementos iguais).
- Ordena in-place, sem necessidade de memória extra — $O(1)$ de espaço adicional.
- Com a otimização de parada antecipada, é eficiente em arrays já quase ordenados.

Limitações:

- Extremamente ineficiente para grandes volumes de dados, devido ao crescimento quadrático.
- Número de comparações e trocas cresce muito rapidamente conforme n aumenta.

Situações em que é adequado: conjuntos pequenos de dados, fins didáticos (por sua simplicidade), ou quando os dados já estão quase ordenados.

Situações em que não é recomendado: grandes volumes de dados ou aplicações em que desempenho é um requisito importante.

Quick Sort

Como funciona / lógica de ordenação:

- Utiliza a estratégia "dividir para conquistar".
- Escolhe um elemento como pivô (pode ser o primeiro, o último, o do meio ou aleatório).
- Particiona o array: elementos menores que o pivô vão para um lado, maiores para o outro; ao final da partição, o pivô já está em sua posição definitiva.
- Aplica o mesmo processo recursivamente às duas sublistas geradas (esquerda e direita do pivô), até que cada sublista tenha 0 ou 1 elemento.

Complexidade — Melhor caso: $O(n \log n)$, quando o pivô sempre divide o array em duas metades aproximadamente iguais.

Complexidade — Caso médio: $O(n \log n)$, comportamento típico para dados aleatórios.

Complexidade — Pior caso: $O(n^2)$, quando a escolha do pivô é sistematicamente ruim (ex.: array já ordenado e pivô sempre no extremo), gerando partições muito desbalanceadas.

Vantagens:

- Extremamente rápido na prática para grandes volumes de dados.
- Ordena in-place, exigindo apenas $O(\log n)$ de memória extra (pilha de recursão).
- É a base de implementações de ordenação em diversas bibliotecas e linguagens de programação.

Limitações:

- Desempenho pode degradar para $O(n^2)$ se o pivô for mal escolhido.
- Não é estável — a ordem relativa de elementos iguais pode ser alterada.
- A recursão pode causar estouro de pilha em casos patológicos ou implementações não otimizadas.

Situações em que é adequado: grandes volumes de dados e aplicações em que desempenho é crítico.

Situações em que não é recomendado: quando estabilidade da ordenação é exigida, ou quando não é possível aleatorizar a escolha do pivô em dados já quase ordenados.

Tabela comparativa

Característica	Bubble Sort	Quick Sort
Princípio de funcionamento	Comparação e troca de elementos adjacentes	Divisão e conquista com particionamento em torno de um pivô
Melhor caso	$O(n)$	$O(n \log n)$
Caso médio	$O(n^2)$	$O(n \log n)$
Pior caso	$O(n^2)$	$O(n^2)$
Uso de memória	$O(1)$ — in-place	$O(\log n)$ — pilha de recursão
Vantagem principal	Simplicidade de implementação	Alta eficiência em grandes volumes de dados
Limitação principal	Ineficiente para grandes entradas	Pior caso quadrático com pivô mal escolhido
Aplicação recomendada	Ensino, arrays pequenos ou quase ordenados	Ordenação de grandes conjuntos de dados

Parte 2 – Experimento de Ordenação

Os dois algoritmos foram implementados em Python, instrumentados para contar comparações e trocas/movimentações. Para cada tamanho de array (10, 20 e 1.000 elementos), o mesmo conjunto de dados aleatório foi copiado e usado nos dois algoritmos, garantindo uma comparação justa.

Código – Bubble Sort

```
def bubble_sort(arr):
    a = arr.copy()
    n = len(a)
    comparacoes = 0
    trocas = 0
    for i in range(n - 1):
        trocou = False
        for j in range(n - 1 - i):
            comparacoes += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                trocas += 1
                trocou = True
        if not trocou:
            break
    return a, comparacoes, trocas
```

Código – Quick Sort

```
def quick_sort(arr):
    a = arr.copy()
    comparacoes = 0
    movimentacoes = 0

    def partition(lo, hi):
        nonlocal comparacoes, movimentacoes
        pivot = a[hi]
        i = lo - 1
        for j in range(lo, hi):
            comparacoes += 1
            if a[j] <= pivot:
                i += 1
                if i != j:
                    a[i], a[j] = a[j], a[i]
                    movimentacoes += 1
        if i + 1 != hi:
            a[i + 1], a[hi] = a[hi], a[i + 1]
            movimentacoes += 1
        return i + 1

    def qs(lo, hi):
        if lo < hi:
            p = partition(lo, hi)
            qs(lo, p - 1)
            qs(p + 1, hi)

    qs(0, len(a) - 1)
    return a, comparacoes, movimentacoes
```

Resultados dos testes (dados reais de execução)

Tamanho do Array	Bubble – Comparações	Bubble – Trocas	Quick – Comparações	Quick – Movimentações
10	45	26	29	10
20	187	114	62	40
1.000	499.329	241.624	10.695	5.144

Tempos de execução medidos (informação adicional, não usada como comparação principal, pois dependem do computador utilizado):

Tamanho do Array	Bubble Sort (ms)	Quick Sort (ms)
10	0,008	0,060
20	0,016	0,012
1.000	46,444	1,066

Respostas

a) Qual algoritmo realizou menos operações para 10 elementos? O Quick Sort: 39 operações no total (29 comparações + 10 movimentações) contra 71 do Bubble Sort (45 comparações + 26 trocas).

b) O comportamento permaneceu igual para 20 elementos? Sim, o Quick Sort continuou realizando menos operações (102 no total) do que o Bubble Sort (301 no total), e a diferença relativa entre os dois já começou a aumentar.

c) O que aconteceu quando o tamanho aumentou para 1.000? A diferença se tornou drástica: o Bubble Sort realizou 740.953 operações no total, enquanto o Quick Sort realizou apenas 15.839 — quase 47 vezes menos.

d) Qual algoritmo apresentou maior crescimento da quantidade de operações? O Bubble Sort, cujo número de operações cresce de forma quadrática ($O(n^2)$) em relação ao tamanho da entrada.

e) Os resultados experimentais são coerentes com as complexidades teóricas estudadas? Sim. O crescimento quase 500x nas comparações do Bubble Sort (de 45 para quase 500 mil) ao passar de $n=10$ para $n=1.000$ é compatível com $O(n^2)$; o crescimento muito mais moderado do Quick Sort é compatível com $O(n \log n)$.

f) Em qual situação você escolheria Bubble Sort? Em conjuntos pequenos de dados, em contextos didáticos, ou quando os dados já estão quase ordenados e a simplicidade da implementação é mais importante que a performance.

g) Em qual situação você escolheria Quick Sort? Em qualquer cenário com grandes volumes de dados e onde desempenho é um requisito relevante — é a escolha padrão para ordenação de propósito geral.

Parte 3 – Investigação de Busca em Matrizes

Foi implementado um algoritmo de busca sequencial com loops aninhados, que percorre a matriz linha por linha e coluna por coluna.

Código – Busca sequencial em matriz

```
def busca_sequencial(matriz, alvo):
    linhas = len(matriz)
    colunas = len(matriz[0])
    comparacoes = 0
    for i in range(linhas):
        for j in range(colunas):
            comparacoes += 1
            if matriz[i][j] == alvo:
                return True, i, j, comparacoes
    return False, -1, -1, comparacoes
```

Resultados dos testes (quantidade de comparações realizadas)

Matriz	Nº de elementos	Busca no início	Busca no final	Valor inexistente
2×2	4	1	3	4
10×10	100	1	99	100
100×100	10.000	1	9.999	10.000

Respostas

a) Por que encontrar um elemento no início exige menos operações? Porque a busca sequencial percorre a matriz na ordem em que os elementos foram organizados (linha a linha); se o valor procurado está na primeira posição, apenas uma comparação é necessária antes que o algoritmo encontre e interrompa a busca.

b) O que acontece quando o elemento procurado não existe? O algoritmo é obrigado a percorrer todas as posições da matriz, sem interrupção antecipada, realizando o número máximo possível de comparações (linhas $\times$ colunas).

c) Qual é o pior caso da busca sequencial? Ocorre quando o valor está na última posição verificada ou é inexistente — em ambos os casos, todas as $m \times n$ posições precisam ser comparadas.

d) Como o aumento das dimensões da matriz influencia a quantidade de operações? O número máximo de comparações cresce proporcionalmente ao total de elementos (linhas $\times$ colunas); ao passar de 10×10 para 100×100 , por exemplo, o número de elementos (e o pior caso de comparações) cresce de 100 para 10.000 — cem vezes mais.

e) Qual a complexidade da busca sequencial em uma matriz $m \times n$? $O(m \times n)$ — complexidade linear em relação ao número total de elementos da matriz.

Parte 4 – Hands On 1: Investigação do Array

Array de 10 temperaturas, processado para calcular média, maior/menor valor (com seus índices) e quantidade de valores acima da média.

Código

```
temperatura = [19.9, 17.3, 24.8, 16.1, 23.0, 20.5, 15.9, 22.6, 15.6, 21.5]

soma = 0
maior = temperatura[0]
menor = temperatura[0]
indice_maior = 0
indice_menor = 0

for i in range(len(temperatura)):
    soma += temperatura[i]
    if temperatura[i] > maior:
        maior = temperatura[i]
        indice_maior = i
    if temperatura[i] < menor:
        menor = temperatura[i]
        indice_menor = i

media = soma / len(temperatura)

acima_media = 0
for i in range(len(temperatura)):
    if temperatura[i] > media:
        acima_media += 1

print("Temperaturas:", temperatura)
print(f"Média: {media:.2f} °C")
print(f"Maior: {maior} °C (índice {indice_maior})")
print(f"Menor: {menor} °C (índice {indice_menor})")
print(f"Valores acima da média: {acima_media}")
```

Resultado da execução

Índice	0	1	2	3	4	5	6	7	8	9
Temp. (°C)	19,9	17,3	24,8	16,1	23,0	20,5	15,9	22,6	15,6	21,5

- Média: 19,72 °C
- Maior valor: 24,8 °C — índice 2
- Menor valor: 15,6 °C — índice 8
- Valores acima da média: 6

Análise de operações e complexidade

O algoritmo realiza 4 percursos completos sobre o array de n elementos: (1) acumular a soma, (2) verificar maior/menor, (3) exibir os elementos e (4) contar quantos valores estão acima da média. Cada percurso realiza n operações, logo:

$T(n) = 4n \rightarrow$ para $n = 10$, $T(10) = 40$ operações (valor confirmado na execução do código instrumentado).

Como a constante 4 é desprezada na notação Big-O, a complexidade do algoritmo é $O(n)$ — crescimento linear em relação ao tamanho do array.

Parte 5 – Hands On 2: Matriz Aplicada – Monitoramento de Sensores

Matriz 5×24 (5 sensores $\times$ 24 medições/hora), usada para calcular médias por sensor, maior temperatura registrada (com sensor e horário), média geral e quantidade de leituras acima de um limite informado.

Código

```
def analisar_sensores(sensores, limite):
    n_sensores = len(sensores)
    n_horas = len(sensores[0])

    media_sensor = [0.0] * n_sensores
    soma_geral = 0.0
    maior_temp = sensores[0][0]
    sensor_maior = 0
    horario_maior = 0
    acima_limite = 0

    for i in range(n_sensores):
        soma_sensor = 0.0
        for j in range(n_horas):
            temp = sensores[i][j]
            soma_sensor += temp
            soma_geral += temp

            if temp > maior_temp:
                maior_temp = temp
                sensor_maior = i
                horario_maior = j

            if temp > limite:
                acima_limite += 1

        media_sensor[i] = soma_sensor / n_horas

    media_geral = soma_geral / (n_sensores * n_horas)
    return media_sensor, maior_temp, sensor_maior, horario_maior, media_geral, acima_limite
```

Resultado da execução (limite = 28,0 °C)

Sensor 0	Sensor 1	Sensor 2	Sensor 3	Sensor 4
23,80 °C	24,06 °C	23,59 °C	23,41 °C	24,28 °C

- Maior temperatura registrada: 30,0 °C
- Sensor responsável: Sensor 1
- Horário da ocorrência: 12h
- Média geral (das 120 medições): 23,83 °C
- Leituras acima de 28,0 °C: 23

Análise final

Por que são necessários loops aninhados: porque a matriz tem duas dimensões (sensores e horários); um único loop percorreria apenas uma linha ou uma coluna. O loop externo percorre cada sensor (linha) e o loop interno percorre cada horário (coluna) daquele sensor, garantindo que todas as $5 \times 24 = 120$ medições sejam visitadas.

Qual o papel dos índices [i][j]: i identifica o sensor (a linha da matriz) e j identifica o horário, de 0 a 23 (a coluna da matriz). Juntos, sensores[i][j] localizam exatamente uma medição específica.

Quantas posições da matriz são percorridas: todas as 120 posições (5 sensores $\times$ 24 horários) são visitadas exatamente uma vez, conforme confirmado pela execução do código instrumentado.

Relação entre linhas, colunas e quantidade de operações: o número de operações é proporcional ao produto do número de linhas pelo número de colunas — $T(m, n) = k \times m \times n$, com complexidade $O(m \times n)$. Aumentar o número de sensores ou de horários monitorados aumenta linearmente o número total de operações.

Parte 6 – Análise e Conclusão

1. O aumento do tamanho da estrutura de dados influencia a quantidade de operações?

Sim. Em todos os experimentos realizados — ordenação, busca em matriz e os dois Hands On — o número de operações cresceu junto com o tamanho da entrada. A taxa desse crescimento, porém, depende da complexidade do algoritmo: no Bubble Sort o crescimento foi quadrático, enquanto no Quick Sort e nos algoritmos de percurso linear (Hands On 1 e 2, busca sequencial) o crescimento foi linear ou próximo disso.

2. Bubble Sort e Quick Sort crescem da mesma maneira quando o número de elementos aumenta?

Não. O Bubble Sort tem complexidade $O(n^2)$: ao multiplicar o tamanho da entrada por 100 (de 10 para 1.000), o número de comparações cresceu cerca de 11.000 vezes (de 45 para quase 500 mil). Já o Quick Sort, com complexidade $O(n \log n)$, teve um crescimento muito mais moderado no mesmo intervalo (de 29 para cerca de 10.700 comparações). Essa diferença de comportamento, imperceptível para arrays pequenos, torna-se decisiva conforme os dados aumentam.

3. Por que analisar somente o resultado final da ordenação não é suficiente para comparar algoritmos?

Porque o resultado final — o array ordenado — é idêntico para os dois algoritmos, mas o caminho até chegar a esse resultado (a quantidade de comparações, trocas e movimentações realizadas) pode ser radicalmente diferente. Observar apenas o resultado esconde informações essenciais sobre eficiência e escalabilidade: um algoritmo pode ser inviável para grandes volumes de dados mesmo produzindo o resultado correto, e isso só fica evidente ao medir e comparar o número de operações — ou, de forma mais geral, a complexidade computacional de cada algoritmo.

Conclusão geral

Os experimentos confirmaram, na prática, que o tamanho da entrada e o algoritmo escolhido determinam juntos o custo computacional de uma tarefa. Estruturas simples como arrays e matrizes oferecem acesso rápido e direto ($O(1)$) às suas posições, mas operações que exigem percorrer todos os elementos (busca, soma, comparação) têm custo proporcional ao tamanho da estrutura — $O(n)$ em arrays e $O(m \times n)$ em matrizes. Entre os algoritmos de ordenação, o experimento evidenciou de forma clara a diferença entre $O(n^2)$ e $O(n \log n)$: para entradas pequenas essa diferença é discreta, mas para entradas maiores ela se torna o fator determinante na viabilidade do algoritmo. A relação central trabalhada em toda a atividade — tamanho da entrada → número de operações → complexidade → eficiência do algoritmo — é, portanto, o critério correto para comparar algoritmos, e não apenas a corretude do resultado final.